204252 Data Structures and Analysis

Chapter 10: Sorting (Part 1)

Lecturer

August 24, 2026

Computer Science Department

Outline

Introduction to Sorting & Stability

Selection Sort

Insertion Sort

Merge Sort

Introduction to Sorting & Stability

The Sorting Problem

Formal Definition:

- **Input**: A sequence of N records $\langle a_1, a_2, \ldots, a_N \rangle$.
- **Output**: A permutation $\langle a'_1, a'_2, \ldots, a'_N \rangle$ of the input such that:

$$a'_1 \leq a'_2 \leq \cdots \leq a'_N$$

Assumptions:

- Elements are stored in an array or list.
- Keys are **Comparable**: a strict weak ordering exists ($<$ is defined).
- We focus on **Internal Sorting**: sorting done in main memory (typically $N < 10^7$).

The Comparison Model

What is the Comparison Model?

- The only operation allowed to gain ordering information between elements is a **pairwise comparison** ($a_i < a_j$, $a_i \leq a_j$, etc.).
- We cannot inspect the internal binary representation of keys (no bitwise operations or hashing).

Significance:

- Any comparison-based sorting algorithm has a fundamental mathematical lower bound on its worst-case complexity:

$$\Omega(N \log N) \text{ comparisons}$$

- We will prove this bound in Part 2 using the **Decision Tree Model**.

Stability in Sorting

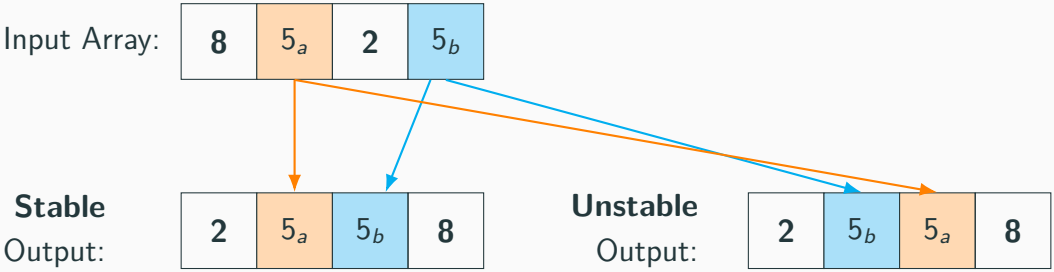
Definition:

- A sorting algorithm is **stable** if it preserves the relative input order of records with equal keys.
- If $key(a_i) == key(a_j)$ and a_i appears before a_j in the input, then a_i must appear before a_j in the sorted output.

Why Stability Matters:

- Useful when sorting by multiple keys (multi-key sorting).
- *Example*: Sort a list of student records by First Name, and then by Last Name. A stable sort preserves the First Name order for students with the same Last Name.

Stability Visualized



- In the stable output, 5_a remains before 5_b.
- In the unstable output, their relative order is reversed.

Selection Sort

Selection Sort: Core Idea

Concept:

- Divide the array into a **sorted** prefix and an **unsorted** suffix.
- In each pass, scan the unsorted suffix to find the **minimum** element.
- Swap this minimum element with the first element of the unsorted suffix.
- Grow the sorted prefix by 1 element.

Key Characteristics:

- Simple and intuitive.
- Performs a **linear** number of swaps ($\mathcal{O}(N)$ swaps).
- Run time is insensitive to input order; it is always $\Theta(N^2)$.
- It is inherently **unstable**.

Selection Sort: C++ Implementation

```cpp
template <typename Comparable>
void selectionSort(vector<Comparable> & a) {
    for (size_t i = 0; i < a.size() - 1; ++i) {
        size_t min_idx = i;
        // Find minimum element in the unsorted suffix
        for (size_t j = i + 1; j < a.size(); ++j) {
            if (a[j] < a[min_idx]) {
                min_idx = j;
            }
        }
        // Swap with the first element of the unsorted suffix
        if (min_idx != i) {
            std::swap(a[i], a[min_idx]);
        }
    }
}
```

Selection Sort: Visual Trace

Sort array: $[5_a, 3, 8, 2, 5_b]$

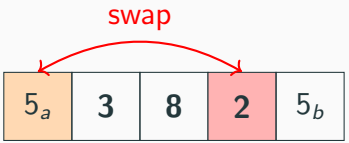
Initial state



Selection Sort: Visual Trace

Sort array: $[5_a, 3, 8, 2, 5_b]$

Pass 0: find min in $[5_a, 3, 8, 2, 5_b]$



Selection Sort: Visual Trace

Sort array: $[5_a, 3, 8, 2, 5_b]$

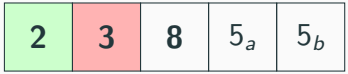
After Pass 0



Selection Sort: Visual Trace

Sort array: $[5_a, 3, 8, 2, 5_b]$

Pass 1: find min in $[3, 8, 5_a, 5_b]$



Min is already at index 1

Selection Sort: Visual Trace

Sort array: $[5_a, 3, 8, 2, 5_b]$

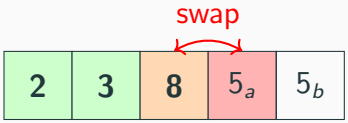
After Pass 1



Selection Sort: Visual Trace

Sort array: $[5_a, 3, 8, 2, 5_b]$

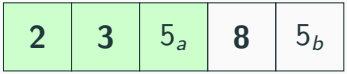
Pass 2: find min in $[8, 5_a, 5_b]$



Selection Sort: Visual Trace

Sort array: $[5_a, 3, 8, 2, 5_b]$

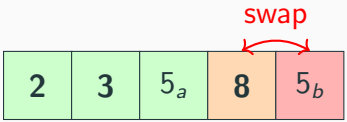
After Pass 2



Selection Sort: Visual Trace

Sort array: $[5_a, 3, 8, 2, 5_b]$

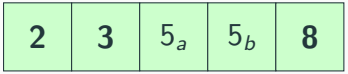
Pass 3: find min in $[8, 5_b]$



Selection Sort: Visual Trace

Sort array: $[5_a, 3, 8, 2, 5_b]$

Sorted Array



Selection Sort: Unstable Counterexample

Why is Selection Sort **unstable**? Let's trace it on $[2_a, 2_b, 1]$:

Initial array:



Selection Sort: Unstable Counterexample

Why is Selection Sort **unstable**? Let's trace it on $[2_a, 2_b, 1]$:



Selection Sort: Unstable Counterexample

Why is Selection Sort **unstable**? Let's trace it on $[2_a, 2_b, 1]$:

After swapping 2_a and 1



Selection Sort: Unstable Counterexample

Why is Selection Sort **unstable**? Let's trace it on $[2_a, 2_b, 1]$:

Final sorted array:



Relative order of 2_a and 2_b is reversed!

Selection Sort: Complexity Analysis

Comparisons:

- Independent of input! To sort N elements, we always scan:

$$(N-1) + (N-2) + \cdots + 1 = \frac{N(N-1)}{2} \text{ comparisons}$$

- Time Complexity: $\Theta(N^2)$ (Best, Worst, and Average case).

Swaps:

- Exactly $N-1$ swaps are performed (at most 1 swap per outer iteration).
- Time Complexity: $\Theta(N)$ swaps.
- *Practical Advantage*: If swapping records is extremely expensive, Selection Sort can be faster than other quadratic sorts because it minimizes writing back to memory.

Insertion Sort

Insertion Sort: Core Idea

Concept:

- Consists of $N-1$ passes (for $p=1$ to $N-1$).
- At pass p, we assume that elements from index 0 to $p-1$ are already sorted.
- We move the element at index p (denoted as 'tmp') leftward until we find its correct position among the first $p+1$ elements.
- Shifting larger elements to the right makes a hole for 'tmp'.

Key Characteristics:

- Simple and highly efficient for small arrays.
- **Adaptive**: runs in $\mathcal{O}(N)$ time if input is already sorted!
- It is a **stable** sorting algorithm.

Insertion Sort: C++ Implementation

Insertion Sort (Weiss Figure 7.2)

```
1  template <typename Comparable>
2  void insertionSort( vector<Comparable> & a ) {
3      for( int p = 1; p < a.size( ); ++p ) {
4          Comparable tmp = std::move( a[p] );
5          int j;
6          for( j = p; j > 0 && tmp < a[j - 1]; --j ) {
7              a[j] = std::move( a[j - 1] );
8          }
9          a[j] = std::move( tmp );
10     }
11 }
```

Insertion Sort: Step-by-Step Trace

Sort array: $[34, 8, 64, 51, 32, 21]$

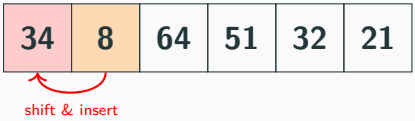
Initial Array



Insertion Sort: Step-by-Step Trace

Sort array: $[34, 8, 64, 51, 32, 21]$

Pass 1: Insert 8



Insertion Sort: Step-by-Step Trace

Sort array: $[34, 8, 64, 51, 32, 21]$

After Pass 1 (sorted prefix: $[8, 34]$)

8	**34**	**64**	**51**	**32**	**21**

Insertion Sort: Step-by-Step Trace

Sort array: $[34, 8, 64, 51, 32, 21]$

Pass 2: Insert 64



No shift ($64 > 34$)

Insertion Sort: Step-by-Step Trace

Sort array: $[34, 8, 64, 51, 32, 21]$

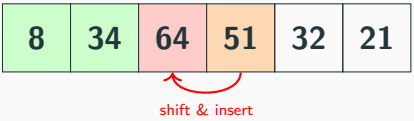
After Pass 2 (sorted prefix: $[8, 34, 64]$)

8	**34**	**64**	**51**	**32**	**21**

Insertion Sort: Step-by-Step Trace

Sort array: $[34, 8, 64, 51, 32, 21]$

Pass 3: Insert 51



Insertion Sort: Step-by-Step Trace

Sort array: $[34, 8, 64, 51, 32, 21]$

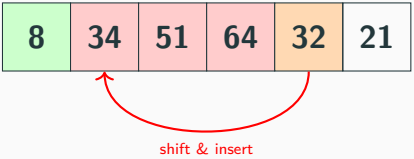
After Pass 3 (sorted prefix: $[8, 34, 51, 64]$)

8	**34**	**51**	**64**	**32**	**21**

Insertion Sort: Step-by-Step Trace

Sort array: $[34, 8, 64, 51, 32, 21]$

Pass 4: Insert 32



Insertion Sort: Step-by-Step Trace

Sort array: $[34, 8, 64, 51, 32, 21]$

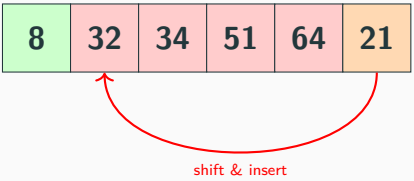
After Pass 4 (sorted prefix: $[8, 32, 34, 51, 64]$)

8	**32**	**34**	**51**	**64**	**21**

Insertion Sort: Step-by-Step Trace

Sort array: $[34, 8, 64, 51, 32, 21]$

Pass 5: Insert 21



Insertion Sort: Step-by-Step Trace

Sort array: $[34, 8, 64, 51, 32, 21]$

Final Sorted Array

8	21	32	34	51	64

Insertion Sort: Complexity Analysis

- **Worst-case**: Reversed input. Inner loop executes p times for each p.

$$\sum_{p=1}^{N-1} p = \frac{N(N-1)}{2} = \mathcal{O}(N^2) \text{ comparisons and shifts}$$

- **Best-case**: Presorted input. Inner loop check ‘tmp $<$ a[j - 1]‘ fails immediately.

$$\mathcal{O}(N) \text{ total comparisons, 0 shifts.}$$

- **Average-case**: $\Theta(N^2)$ comparisons and shifts.

Insertion Sort: Complexity Analysis

Inversion Relationship:

- An **inversion** is a pair (i, j) such that $i < j$ but $a[i] > a[j]$.
- Each shifting step in Insertion Sort removes exactly **one inversion**.
- Running time is exactly:

$$\mathcal{O}(I + N)$$

where I is the number of inversions.

Inversion Lower Bound Theorem

Theorem: The average number of inversions in an unsorted array of N distinct numbers is:

$$I_{\text{avg}} = \frac{N(N-1)}{4}$$

Proof:

- Consider any list L and its reverse list L_r (e.g. $L = [3, 4, 1, 5, 2]$, $L_r = [2, 5, 1, 4, 3]$).
- For any pair of elements (x, y), it is an inversion in exactly one of L or L_r.
- The total number of pairs in a list of size N is:

$$\frac{N(N-1)}{2}$$

- Thus, the sum of inversions in L and L_r is exactly $\frac{N(N-1)}{2}$.
- By symmetry, a random list has half that amount on average: $\frac{N(N-1)}{4} = \Theta(N^2)$.
- **Corollary**: Any sorting algorithm that swaps adjacent elements must run in $\Omega(N^2)$ average time.

Merge Sort

Merge Sort: Core Idea

Concept:

- A classic **Divide-and-Conquer** algorithm.
- **Divide**: Split the array into two equal halves.
- **Conquer**: Recursively sort each half.
- **Combine**: Merge the two sorted halves into a single sorted array.

The Merge Operation:

- Takes two sorted subarrays and combines them in linear ($\mathcal{O}(N)$) time.
- Requires an auxiliary temporary array of size N.
- Compares elements from both halves; copies the smaller element and advances pointers.

Merge Sort: Driver and Recursive C++ Code

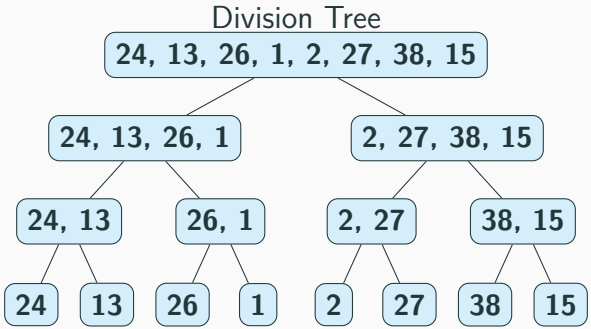
```
1  // Public driver
2  template <typename Comparable>
3  void mergeSort( vector<Comparable> & a ) {
4      vector<Comparable> tmpArray( a.size( ) );
5      mergeSort( a, tmpArray, 0, a.size( ) - 1 );
6  }
7
8  // Internal recursive method
9  template <typename Comparable>
10 void mergeSort( vector<Comparable> & a, vector<Comparable> & tmpArray,
11                 int left, int right ) {
12     if( left < right ) {
13         int center = ( left + right ) / 2;
14         mergeSort( a, tmpArray, left, center );
15         mergeSort( a, tmpArray, center + 1, right );
16         merge( a, tmpArray, left, center + 1, right );
17     }
18 }
```

Merge Sort: Merge C++ Implementation

```
1  template <typename Comparable>
2  void merge( vector<Comparable> & a, vector<Comparable> & tmpArray,
3              int leftPos, int rightPos, int rightEnd ) {
4      int leftEnd = rightPos - 1;
5      int tmpPos = leftPos;
6      int numElements = rightEnd - leftPos + 1;
7      while( leftPos <= leftEnd && rightPos <= rightEnd )
8          if( a[ leftPos ] <= a[ rightPos ] )
9              tmpArray[ tmpPos++ ] = std::move( a[ leftPos++ ] );
10         else
11             tmpArray[ tmpPos++ ] = std::move( a[ rightPos++ ] );
12     while( leftPos <= leftEnd )     // Copy rest of first half
13         tmpArray[ tmpPos++ ] = std::move( a[ leftPos++ ] );
14     while( rightPos <= rightEnd )   // Copy rest of second half
15         tmpArray[ tmpPos++ ] = std::move( a[ rightPos++ ] );
16     for( int i = 0; i < numElements; ++i, --rightEnd ) // Copy back
17         a[ rightEnd ] = std::move( tmpArray[ rightEnd ] );
18 }
```

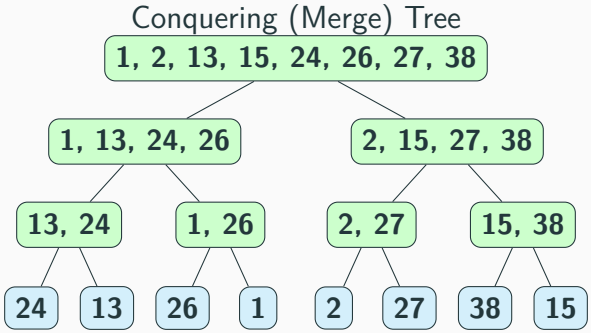
Merge Sort: Recursive Traces

Sort array: $[24, 13, 26, 1, 2, 27, 38, 15]$



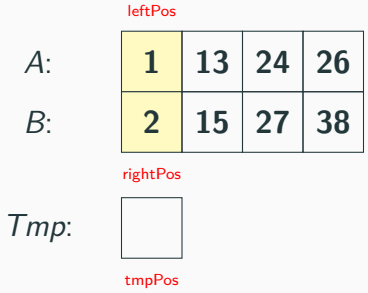
Merge Sort: Recursive Traces

Sort array: $[24, 13, 26, 1, 2, 27, 38, 15]$



Merge Operation: Detail Trace

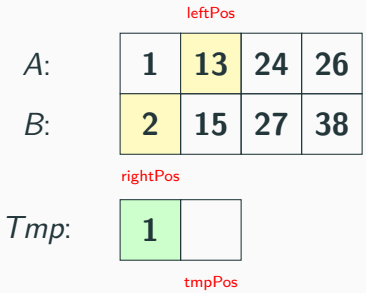
Merge sorted halves: $A = [1, 13, 24, 26]$ and $B = [2, 15, 27, 38]$



Compare 1 **and** 2: $1 \leq 2$

Merge Operation: Detail Trace

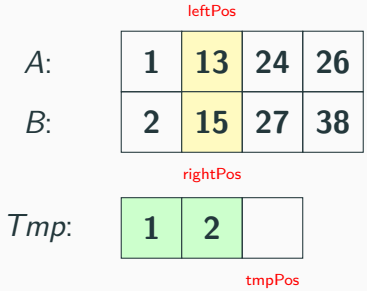
Merge sorted halves: $A = [1, 13, 24, 26]$ and $B = [2, 15, 27, 38]$



Compare 13 **and** 2: $2 < 13$

Merge Operation: Detail Trace

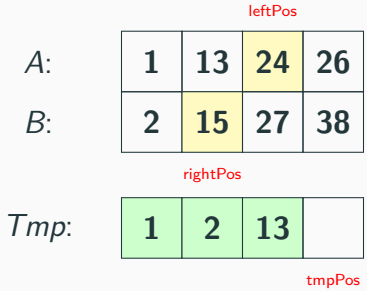
Merge sorted halves: $A = [1, 13, 24, 26]$ and $B = [2, 15, 27, 38]$



Compare 13 **and** 15: $13 \leq 15$

Merge Operation: Detail Trace

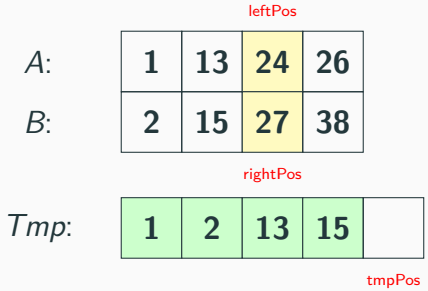
Merge sorted halves: $A = [1, 13, 24, 26]$ and $B = [2, 15, 27, 38]$



Compare 24 **and** 15: $15 < 24$

Merge Operation: Detail Trace

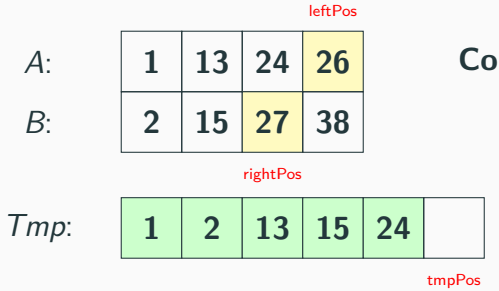
Merge sorted halves: $A = [1, 13, 24, 26]$ and $B = [2, 15, 27, 38]$



Compare 24 **and** 27: $24 \leq 27$

Merge Operation: Detail Trace

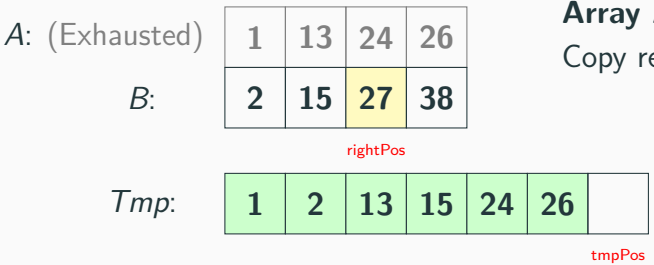
Merge sorted halves: $A = [1, 13, 24, 26]$ and $B = [2, 15, 27, 38]$



Compare 26 **and** 27: $26 \leq 27$

Merge Operation: Detail Trace

Merge sorted halves: $A = [1, 13, 24, 26]$ and $B = [2, 15, 27, 38]$



Array A is exhausted!

Copy remaining elements of B.

Merge Operation: Detail Trace

Merge sorted halves: $A = [1, 13, 24, 26]$ and $B = [2, 15, 27, 38]$

A: (Exhausted)	1	13	24	26
B: (Exhausted)	2	15	27	38

Merge Complete!

Copy *Tmp* back to original array.

Tmp:	1	2	13	15	24	26	27	38

Merge Sort Analysis: Recurrence Relation

Assuming N is a power of 2, the running time recurrence relation for Mergesort is:

$$T(1) = 1$$
$$T(N) = 2T(N/2) + N$$

where:

- $2T(N/2)$ is the time to recursively sort the two halves.
- N is the linear time to merge the two sorted halves.

We will solve this recurrence using two separate methods:

1. **Method 1: Telescoping**
2. **Method 2: Substitution**

Recurrence Proof: Method 1 (Telescoping)

Divide both sides of the recurrence by N:

$$\frac{T(N)}{N} = \frac{T(N/2)}{N/2} + 1$$

This relation holds for any power of 2, so we can write it for smaller divisions:

$$\frac{T(N/2)}{N/2} = \frac{T(N/4)}{N/4} + 1$$
$$\frac{T(N/4)}{N/4} = \frac{T(N/8)}{N/8} + 1$$
$$\vdots$$
$$\frac{T(2)}{2} = \frac{T(1)}{1} + 1$$

Summing all these equations cancels (telescopes) intermediate terms:

$$\frac{T(N)}{N} = \frac{T(1)}{1} + \log N \implies T(N) = N \log N + N = \mathcal{O}(N \log N)$$

Recurrence Proof: Method 2 (Substitution)

Substitute $T(N/2) = 2T(N/4) + N/2$ back into the original equation:

$$T(N) = 2[2T(N/4) + N/2] + N = 4T(N/4) + 2N$$

Now substitute $T(N/4) = 2T(N/8) + N/4$:

$$T(N) = 4[2T(N/8) + N/4] + 2N = 8T(N/8) + 3N$$

Continuing this substitution pattern for k steps:

$$T(N) = 2^k T(N/2^k) + kN$$

To terminate the recursion at $T(1)$, we choose $2^k = N$, which means $k = \log N$:

$$T(N) = NT(1) + N\log N = N\log N + N = \mathcal{O}(N\log N)$$

Merge Sort: Complexity and Properties

Time Complexity:

- Worst, Best, and Average case: $\Theta(N \log N)$.
- Highly consistent performance, independent of the initial order.

Space Complexity:

- $\mathcal{O}(N)$ auxiliary space (due to ‘tmpArray‘).
- *Drawback*: Significant memory overhead compared to in-place sorts. Not ideal if memory is constrained.

Stability:

- **Stable**. During merge, if elements are equal, we choose the one from the left subarray first (‘a[leftPos] <= a[rightPos]‘), which preserves their relative order.